

Lab: Midpoint Circle Drawing Algorithm

Objective

To implement the Midpoint Circle Drawing Algorithm using Python and Matplotlib and understand how midpoint decision logic and symmetry are used to draw circles on a raster display.

Introduction

The Midpoint Circle Drawing Algorithm is an efficient technique used in computer graphics to draw circles on raster-based displays. Instead of calculating exact circle points using floating-point arithmetic, this algorithm determines the next pixel position by evaluating a midpoint decision parameter and exploiting eight-way symmetry.

Algorithm Overview

The algorithm starts from the topmost point of the circle and incrementally determines the next pixel based on a decision parameter.

Step-by-Step Algorithm

1. Start the algorithm
2. Declare variables:
 - x_c, y_c : center coordinates
 - r : radius
 - x, y : current point
 - d : decision parameter
3. Input center (x_c, y_c) and radius r
4. Initialize:
 - $x=0$
 - $y=r$
 - $d=1-r$
5. Plot the initial eight symmetric points
6. While $x < y$, repeat:
 - If $d < 0$:

$$d = d + 2x + 3$$
 - Else:

$$d = d + 2(x - y) + 5$$
 and decrement y by 1
 - Increment x by 1
 - Plot eight symmetric points
7. Stop the algorithm

Exercises

1. Draw a circle with center (10, 10) and radius 6 using the Midpoint Circle Algorithm.
2. Modify the program to take center coordinates and radius as user input.
3. Draw multiple circles with the same center but radii 6, 10, and 14.
4. Shift the circle center to (20, 25) and redraw the circle.
5. Display x, y, and decision parameter values at each iteration.
6. Write a Midpoint Circle Drawing program where:
 - Even-numbered iterations plot points
 - Odd-numbered iterations skip plotting

After execution, draw the final corrected circle.

Python Implementation (Using Matplotlib)

```
import matplotlib.pyplot as plt

def plot_circle_points(xc, yc, x, y):
    points = [
        (xc + x, yc + y), (xc - x, yc + y),
        (xc + x, yc - y), (xc - x, yc - y),
        (xc + y, yc + x), (xc - y, yc + x),
        (xc + y, yc - x), (xc - y, yc - x)
    ]
    for px, py in points:
        plt.scatter(px, py)

def midpoint_circle(xc, yc, r):
    x = 0
    y = r
    d = 1 - r
    plot_circle_points(xc, yc, x, y)

    while x < y:
        x += 1
```

```
if d < 0:
    d = d + 2 * x + 1
else:
    y -= 1
    d = d + 2 * (x - y) + 1
plot_circle_points(xc, yc, x, y)
print(f"x={x}, y={y}, d={d}")
```

```
midpoint_circle(10, 10, 6)
plt.gca().set_aspect('equal')
plt.grid(True)
plt.show()
```

Conclusion

This lab demonstrates how midpoint-based decision logic and symmetry enable efficient circle drawing on raster displays.